User Guide

WebFlowMaster Enterprise Documentation

Language: EN | Version 1.0.0

Generated on: 28/04/2026

📖 WebFlowMaster: Advanced User Guide

> **Strategies and Operations for Modern Quality Assurance**

🎯 Overview

WebFlowMaster is not just an automation tool; it is a complete ecosystem for **Continuous Testing**. This guide will take you from creating your first test to managing complex regression suites in enterprise environments.

📑 Table of Contents

1. Fundamentals of Test Automation
2. Mastering the Visual Builder
   - Selector Management (CSS & XPath)
   - Variables and Data Injection
   - Wait Strategies
3. Advanced Recording with the Live Engine
4. API Validation & Integration Testing
5. Strategic Results Analysis
   - Visual Regression Check
   - Network Trace Debugging
6. Best Practices for Robust Testing

---

1. 🚀 Fundamentals of Test Automation

In WebFlowMaster, every test consists of a **Test Sequence**. A sequence is a chain of atomic actions executed in a headless or headful browser. **Key Point:** An effective test must be *deterministic*. The same test, run 100 times without code changes, must produce the same result.

---

2. 🛠️ Mastering the Visual Builder

The Visual Builder is the heart of the platform. it allows you to visualize the logic flow of the test as a directional graph.

A. Selector Management (CSS & XPath)

WebFlowMaster uses advanced algorithms to identify elements. However, for robust enterprise testing, follow these rules:

- **Static IDs**: Always use IDs if present (`#submit-button`).
- **Data Attributes**: Best practice is to use dedicated test attributes like `[data-testid="login-btn"]`.
- **Dynamic XPath**: Use XPath only when you need to navigate the DOM hierarchy relatively (e.g., "find the button next to the text 'Email'").

B. Variables and Data Injection

You can make your tests dynamic using variables:

- `{{ENV_URL}}` : URL of the selected environment.
- `{{RANDOM_EMAIL}}` : Automatically generates a unique email for registration tests.
- **Custom Parameters**: Pass custom parameters during test execution to test different scenarios with the same flow.

C. Wait Strategies

The most common mistake in testing is a lack of synchronization. WebFlowMaster offers three types of waits:

1. **Implicit Wait**: The system waits for the element to be present in the DOM.
2. **Explicit Wait**: Forces a wait of X milliseconds (use with caution).
3. **Condition Wait**: Waits until a specific condition is true (e.g., a loading screen disappearing).

---

3. 📹 Advanced Recording with the Live Engine

The recording engine doesn't just capture clicks. It captures **intent**.

- **Hover Actions**: Move the mouse slowly to capture pop-up menus.
- **Keyboard Events**: Full support for Tab, Enter, and key combinations.
- **Smart Detection**: During recording, the system automatically suggests assertions based on the clicked element.

---

4. 🌐 API Validation & Integration Testing

The API module allows you to test the "core" of your applications.

- **Chaining**: Use the result of an API call (e.g., an authentication Token) as input for the next step.
- **Schema Validation**: Upload a JSON schema to instantly validate that the server response is structurally correct.
- **Status Assertions**: Define acceptable ranges of HTTP codes (e.g., 200-204).

---

5. 📊 Strategic Results Analysis

A. Visual Regression Check

WebFlowMaster compares the current interface with a "Baseline". If a CSS change moves a button by even 5 pixels, the test will trigger a **Warning**. This is critical for preventing unexpected aesthetic regressions.

B. Network Trace Debugging

Each report includes a **"Network Trace"** tab. If a test fails, you can see if an underlying API call timed out or returned a 500 error, immediately identifying whether the problem is in the frontend or backend.

---

6. 🏆 Best Practices for Robust Testing

1. **Atomicity**: Each test should verify a single functionality.
2. **Independence**: A test should not depend on the success of a previous test.
3. **Cleanup**: Ensure the test cleans up created data (or uses isolated environments).
4. **No Flakiness**: If a test fails intermittently, analyze loading times and optimize wait strategies.

© 2026 WebFlowMaster Enterprise. All rights reserved.